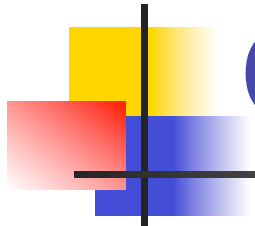


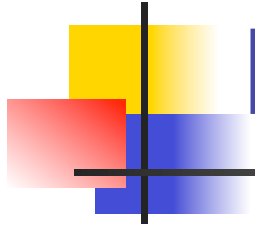
Introduction to Python

Scripting and OOP
Michelle Shaw, CRC
June 24, 2009



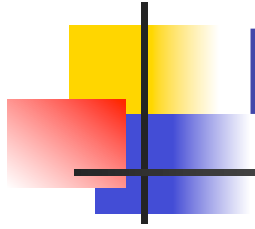
Outline

- Python scripting fundamentals:
 - Program structure
 - Variables, loops, functions
 - modules
- Examples:
 - Start with command line
 - Writing python scripts
 - modules



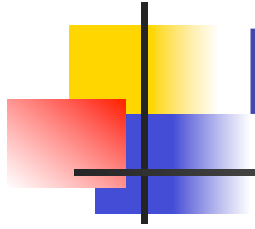
Program structure

- Python is an object-oriented scripting language:
 - Everything falls into two main categories: an object or a handle to the object
 - Structure is very modular, i.e. “objects” and their behavior are grouped together into subprograms like libraries
 - Main program uses these subprograms



Program structure

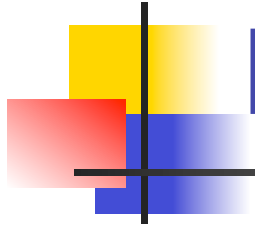
- Python is a scripting language, i.e. it hides much of the deeper program constructs from the programmer
 - E.g. managing memory via malloc/realloc/free
- Less cluttered than some other scripting languages (like perl)
- Requires one to think differently about the construction of their script (OOP)



Program structure

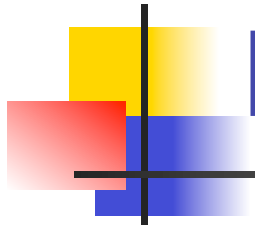
- General rules:
 - Many statements look like this:

Header statement(s):
indented and nested block of statements
 - don't need to include parentheses or end of line characters (; in C)
 - When done with the block, remove the indenting
 - Semicolons can still be used to put multiple statements on one line
 - Line continuation character is a "\"



Program structure

- Python is constructed from modules, so many things not built-in can be accessed by importing the right module, then accessing its functions using:
`module_name.function`
- Python has excellent handling of exceptions through the try-except construct
- Statements written in scripts or at a python command prompt



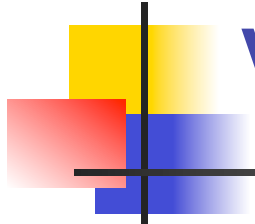
Printing...

- Simple printing is similar to echo in a script or print in perl

print "hello world!"

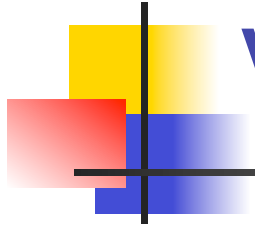
- No newline character, no semicolon
- Also, unformatted...
- Python uses C-type formatted printing:

***print '%02d-%02d: %s' % (n.start(), n.end(),
n.group(0))***



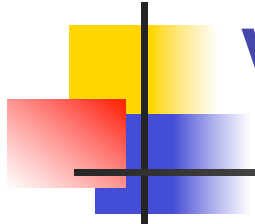
Variables

- “variables” – built-in (or core) data types in python
- They are:
 - Numbers (integer, float, complex)
 - Character sequences
 - Files
 - Lists
 - Dictionaries
 - Tuples



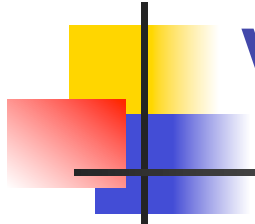
Variables

- Important – some of these can be changed in-place (mutable) and some cannot
- Mutable types:
 - Numbers, strings, tuples
- Immutable:
 - Lists, dictionaries
- Immutability becomes useful when you don't want something to change



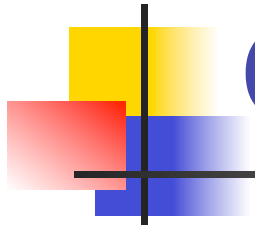
Variables

- An important feature of the OOP way of coding:
 - Variables are not objects, they are only references to an object
 - The object itself is the thing with the type, not the variable
 - Variables don't need to be typed as in traditional programming languages because they don't have a type



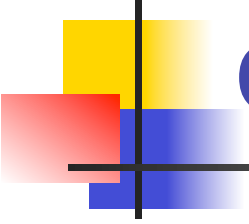
Variables

- Variables act as addresses to an object
- The object contains:
 - The thing it is meant to be (i.e. a number or character)
 - indicator of its type (int, string, etc.)
 - number of references to it
- Important is the last item – it is how python handles cleanup of its memory automatically



Operators

- Various types of built-in operators in python:
 - `+, -, *, /, %, //`
 - `<<, >>, &, |, ^`
 - `<, <=, >, >=, ==, !=`
 - `-x, +x, ~x, x**y`
- Not an exclusive list...
- Remember – operators have different actions for different data types



Examples – variables, operators, printing, etc.

```
a = 1
```

```
b = 2
```

```
print a,b
```

```
print "a is ",a," and b is ",b
```

```
****
```

```
print "multiplication: a*b is ",a*b
```

```
print "addition: a+b is ",a+b
```

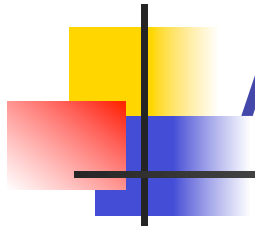
```
print "multiplication: a*b is ",a*b
```

```
print "division: a/b is ",a/b
```

```
c = 2.0
```

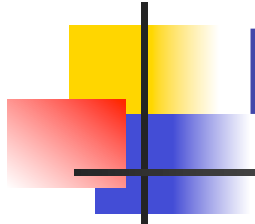
```
d = 3.0
```

```
print "division of floating points: c/d is ",c/d
```



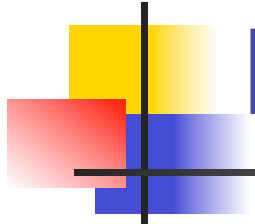
Arrays, Lists and Dictionaries

- Arrays in python are actually handled as lists (sequential) and dictionaries (key-based)
- Unlike traditional languages, they can hold more than one data type
- Note - there is a more traditional array type implemented in numpy (more about this later...)



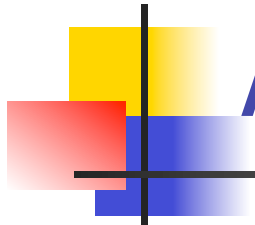
Lists

- Allows one to collect objects of different types:
 - `L = [1.2345, 'junk']`
 - `'junk'` is a 4-character string
 - `1.2345` is a floating point number
- operations on an entire list look similar to those of a string (i.e. `+` concatenates lists)
- On individual elements it depends on the object's type



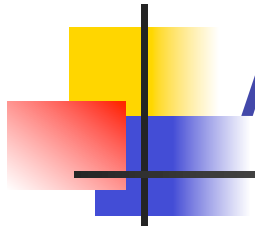
Lists

- NOTE – lists are not true arrays
- To have a list of truly integer/float numbers, we need to use numpy's array type
- For now, we will use lists as collections of different object types (similar, but simpler in syntax, to C's "struct" and fortran's "typedef")



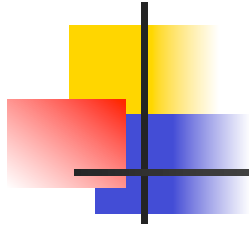
Arrays

- Access to Numpy's array type and tools can be done in one of 2 ways:
 - `import numpy`
 - `from numpy import *`
- Details on those 2 statements to come...
- Allows you to create arrays, zero them, convert numeric lists to array types, etc.



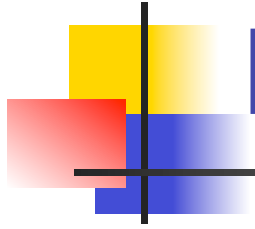
Arrays

- Some useful tools:
 - Array
 - Zeros
 - Arrayrange
 - `<var>.shape`
 - Array operations (`*`, `+`, `-`, `/`)
- NOTE – array operations act on the entire array (so you DON'T need loops!)



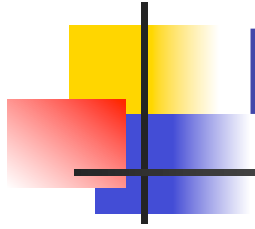
Dictionaries

- Collections of objects indexed by keys
- Similar to a very simple DB:
 - Records indexed and accessed by keys
 - Can do key sorts, searches, etc.
- Syntax:
 - $A = \{\text{'key1': 'e1', 'key2': 'e2', ...}\}$
- Like lists, dictionaries are mutable, i.e. their elements can be changed in-place



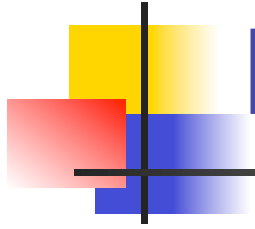
Examples – Lists

```
print "-----LISTS-----"  
abcd = ["alan", "bill", "chris", "dennis"]  
print "index 0: ",abcd[0]  
print "index 1: ",abcd[1]  
print "slice 2-3: ",abcd[2:]
```



Examples – More on Lists

```
abcd.append("eileen")
print "list is now:"
print "all: ",abcd[:]
print "last: ",abcd[-1]
print "first: ",abcd[-5]
abcd.extend(["fred", "greg"])
print "extend by two elements gives: ",abcd
abcd.append(["harry", "irene"])
print "append by two elements gives: ",abcd
abcd.remove(["harry", "irene"])
print "let's remove the second list...",abcd
```



Examples - Dictionaries

```
# third - dictionaries
```

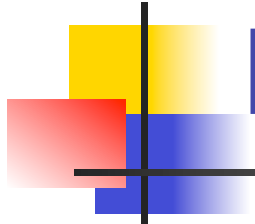
```
authpairs = {"user": "mshaw", "password": "5551234"}
```

```
print "username is: ",authpairs["user"]
```

```
print "passphrase is: ",authpairs["password"]
```

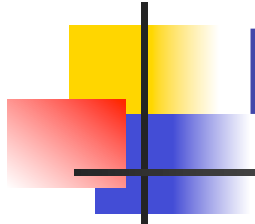
```
bookfaves = {"mystery": "Cromwell", "horror": "King", "comedy":  
             "Dilbert", "drama": "none yet"}
```

```
print "the favorite comedy series is: ",bookfaves["comedy"]
```



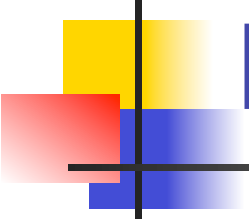
Loops

- Python includes the same loop types as are seen in traditional programming and scripting languages
 - If-elseif
 - While
 - For
- Syntax:
 - Loop header:
Indented statements



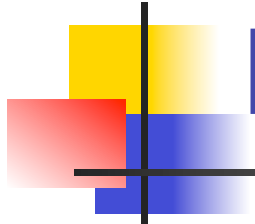
Loops

- Loops are used to iterate over a set of repetitious tasks or do actions based on a set of conditions
- For, while loops are used for iterations
- If loops are used to test conditions
- For loops look similar to those used in shell scripts – we don't need the numerical iterator as python takes care of that for us



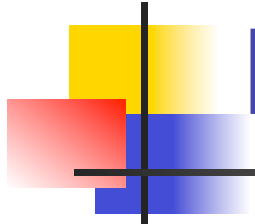
Examples - loops

- If:
 - If <some statement is true>:
Do some actions
- For:
 - For <item> in <list>:
Do some actions
- While:
 - While <some test is true>:
Do some actions



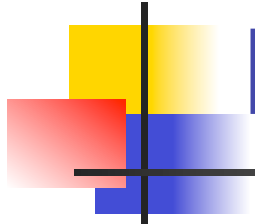
Functions

- Functions in python look similar to other scripting languages
- Defined by starting the line with “def” and following with the function name
- Use of the function is done by putting the function name and any arguments in its round braces



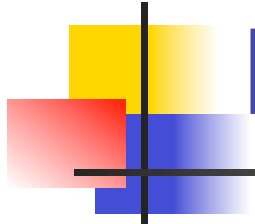
Functions

- Functions are used to group actions that have something in common, i.e. the steps in taking the transpose of a matrix
- Promotes code reuse and readability
- Syntax:
 - def <function name> (arguments):
 - Do some actions
 - Return something (optional)



Functions

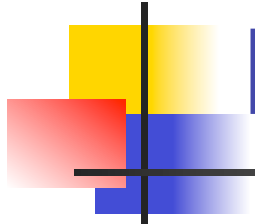
- Use the function by calling it:
 - `X = function(arguments)`
 - `function(arguments)`
- Create your own functions or use those already in python's extensive module library



Modules

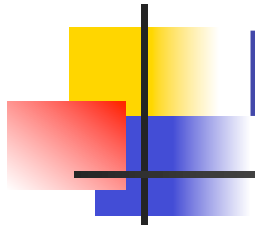
- Modules are a handy way to collect functions together that have a common purpose
- Once defined, we can use them by putting at the top of our python program:

```
import module  
from module import ...  
reload(module)
```



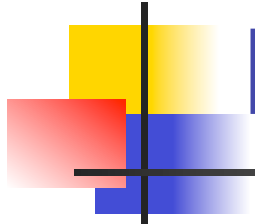
Modules

- Python has a pre-defined path for finding modules
- If you develop your own, you need to do one of two things to get python to see those:
 - Append it to the "PYTHONPATH" environment variable
 - Use the system utility "sys.path.append" in your python program (before importing it) to append it to the python module search path



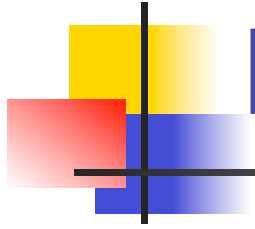
Modules

- Modules need to have a .py extension on the file that contains them
- However, the extension is not needed for the import statement
- Once imported, the attributes (or things the module can do) are accessed by:
 `module.function()`
 OR `function()`
- Usage depends on how the module was imported (wholly or only in parts)



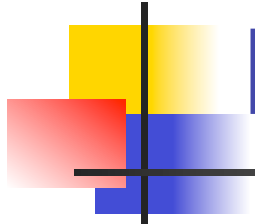
Modules

- To create the module, create a file called `<modulename>.py`
- Then you can import it with the import statements shown earlier
- If you have a module called `mod1` and a function in it called `f1`, then you can use the function by `mod1.f1(...)` or just `f1(...)` if you used “`from mod1 import f1`” to import it



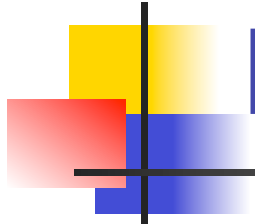
Examples - modules

- Examples of importing functions:
`import mymodule`
`from math import *`
`from numpy import *`
- Differ in the way that they can be used:
 - First: `mymodule.myfunc()`
 - Second and third: `sin(pi)`, `array([9,11,25])`



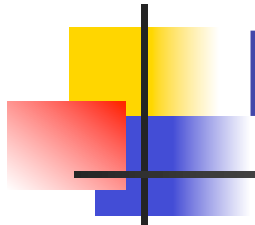
File I/O

- A few types of I/O in python:
 - STDOUT/STDIN read/write
 - File read/write
 - Print to screen
- We've handled printing, start with I/O to STDIN/STDOUT
- Big difference with perl/bash: python is based on objects



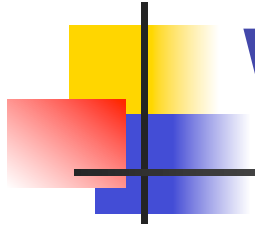
File I/O

- Files in general are objects in python with methods that are specific to them
- File I/O is handled within python normally
- The exceptions – STDIN, STDOUT, STDERR
- To access these objects and their methods, import the “sys” module



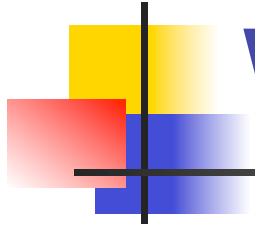
File I/O

- Reading from STDOUT:
 - `raw_input("message:")` – writes "message" to screen then waits for user to input value
 - `stdin.readline()` – reads line from STDOUT
- Writing to STDOUT – use `'print'`, though you can use `stdout.write(value)` to write value to STDOUT



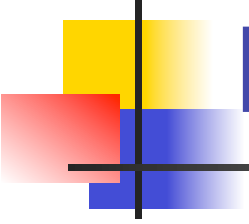
Working with Files

- Functions for working with files in general also in sys module
- Functions applied to the object so use the notation `<obj>.<fn>`
- Some useful functions:
 - `f=open(<file in ">, <mode in ">)`
 - `f.read()` – entire file or `<size>` lines



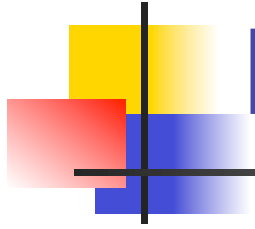
Working with Files

- `f.readline()` – line-by-line read (with `\n` at end of each line except last line)
- `f.write(value)` – write value to file referenced by `f`
- `f.seek(n)` move to byte number `n+1` in the file
- `f.close()` – close file



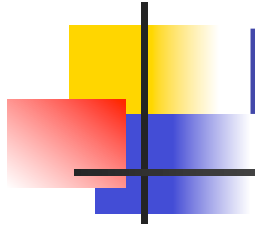
More on Numpy Arrays

- NOTE: “arrays” are not equivalent to “matrices”...
 - Essentially, $A*B$ for A, B defined using `array()` is an element-by-element multiply (i.e. multiplication of 2 arrays)
 - To do a matrix multiply, you need to define the “arrays” using `matrix()`
 - Therefore, if you want to do work involving matrices, use `matrix()` instead of `array()`



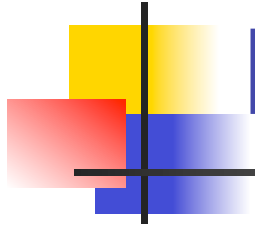
More on Numpy Arrays

- Some useful functions:
 - `A = zeros((m,n),type)` where `m,n` are dimensions and `type` refers to the types of numbers that the array will store (i.e. `int`, `float`, etc.)
 - `X = array([c1,c2,...,cn],...,[cm1,cm2,...,cmn])` creates an `mxn` array with entries `c1, c2, ...`
 - `Y = matrix([c1,...,cn],...,[cm1,...,cmn])` creates the analogous `mxn` matrix



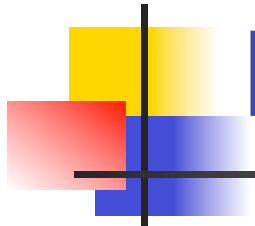
More on Numpy Arrays

- `X.T` is the transpose of `X` (a method that acts as the “behavior” of a particular object)
- `X.I` is the inverse of `X` (not sure of the algorithm...)
- `cross(v1,v2)` is the cross product of vectors `v1` and `v2`
- `linalg.solve(A,x)` solves this system of equations (not sure the algorithm used, though)



More on Numpy Arrays

- `dot(a,b)` – dot product of a and b
- `arange(i1,in,delta)` – make array of integers, numbered i1 to in in increments of delta
- `A.shape = (m,n)` – make A an mxn array
- `size(A)` – number of elements in A



Further reading/help...

- python.org
- Online – plenty of docs on python
- Some good books:
 - O'Reilly – Learning Python, Python Cookbook, Programming Python, Python for Unix and Linux System Administrators
 - Python Scripting for Computational Science
- Ask us: support@ace-net.ca